

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]

Extended by: Taylor Kearney

FIT3155: Algorithms and Data Structures

Week 11 Lecture:

Min-cost matching on bipartite Graphs

Faculty of Information Technology, Monash University

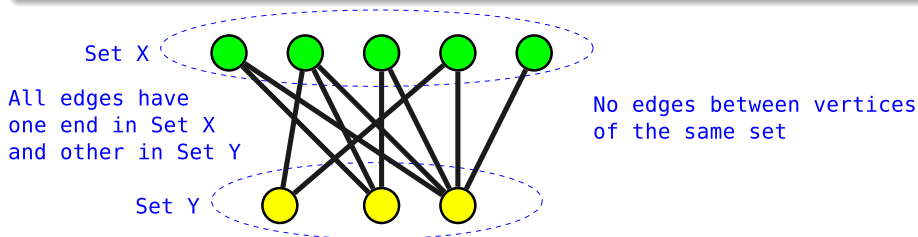
What is covered in this lecture?

- Optimal (minimum-cost) matching on **weighted** bipartite graphs

Bipartite graph

Definition

A graph $G = (V, E)$ is a **bipartite graph** if its set of vertices V can be partitioned into two **disjoint independent subsets** $X \subset V$ and $Y \subset V$, such that each edge $e \in E$ of the graph has one end point (vertex) in X and another (vertex) in Y .

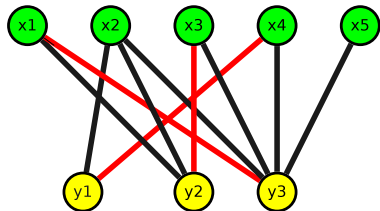


Matching in a bipartite graph

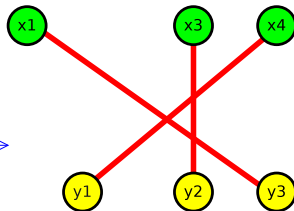
Matching

A **matching** in a bipartite graph $G = (V, E)$ defines another bipartite graph $M = (V' \subseteq V, E' \subseteq E)$ such that **no two edges in E' share the same endpoint (vertex)**. The **cardinality** $|M|$ (i.e. size) of any matching is simply $\frac{|V'|}{2}$.

Bipartite graph $G=(V,E)$
(matched edges in red)



Matching $M=(V',E')$
(In this example,
 $|V'| = 6$ and hence $|M| = 3$)

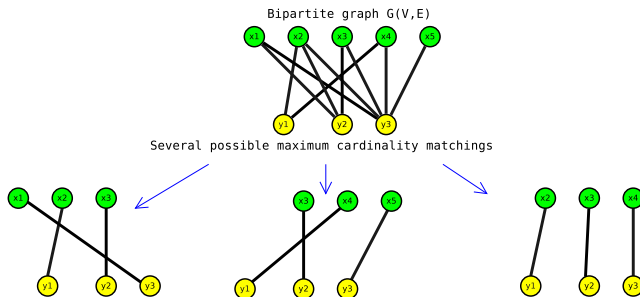


Maximum-cardinality bipartite matching

Maximum-cardinality matching

A **maximum-cardinality** matching in a bipartite graph $G = (V, E)$ is a matching $M = (V' \subseteq V, E' \subseteq E)$ that results in the largest cardinality (i.e. set-size) of the matched edges E' .

Such a set E' need not be unique. That is, depending on the bipartite graph, there can be several ways one could match its vertices, all resulting in the same (maximum) size of E'



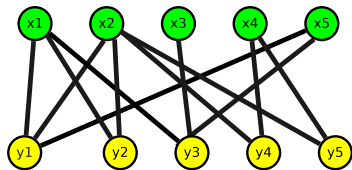
Perfect matching in a bipartite graph

Perfect matching

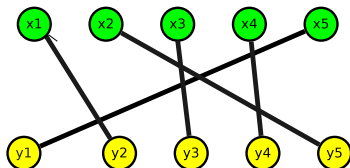
A **perfect** matching in a bipartite graph $G = (V, E)$ is a matching $M = (V', E')$ that covers every vertex of G . That is, $V' = V$ and $|E'| = \frac{|V|}{2}$.

This implies, $|V'| = |V| = 2|X| = 2|Y|$, where $V = \langle X, Y \rangle$ are the two disjoint independent vertex subsets of V that define the bipartition of graph G . Also, a perfect matching need not be unique.

Bipartite graph $G(V, E)$



A perfect matching in G



Minimum-cost bipartite graph matching

Minimum-cost bipartite graph matching

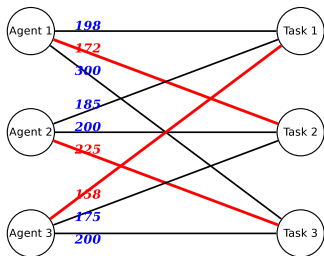
- This problem is also called the **linear assignment problem**.
- The linear assignment problem is among the fundamental optimization problems that frequently arises in real-world settings.
 - ▶ Assigning agents to tasks
 - ▶ Matching students to projects
 - ▶ Assigning nurses to shifts
 - ▶ Tracking and matching objects to features
 - ▶ Allocating bandwidth to telecom users
 - ▶ etc.

Problem statement (standard form)

Statement

Let there be N **agents** and N **tasks**, where any **agent** can be assigned (i.e., matched) to perform any **task** under some **cost**. The problem then is to

- perform **all** N **tasks**,
- assign **exactly one** **agent** to each **task**,
- assign **exactly one** **task** to each **agent**,
- minimize the **total assignment cost**.



Cost matrix with min-cost assignment

	Task 1	Task 2	Task 3
Agent 1	198	172	300
Agent 2	185	200	225
Agent 3	158	175	200

Matching = (1, 2), (2, 3), (3, 1)

Assignment cost = $172 + 225 + 158 = 555$

Formalization of the problem

- Let $x_{i,j}$ denote a binary decision variable, where $x_{i,j} = 1$ if Agent i is assigned to Task j , and $x_{i,j} = 0$ otherwise.
- Let $c_{i,j}$ be the cost of assigning Agent i to Task j .

Formulation

Given an $N \times N$ cost matrix, find a **perfect matching** (of size N) that **minimizes** the total assignment cost

$$z = \sum_{i=1}^N \sum_{j=1}^N c_{i,j} x_{i,j}$$

subject to the constraints:

$$\sum_{i=1}^N x_{i,j} = 1, \quad \forall 1 \leq j \leq N \quad (\text{Each Task is assigned exactly one Agent})$$

$$\sum_{j=1}^N x_{i,j} = 1, \quad \forall 1 \leq i \leq N \quad (\text{Each Agent is assigned exactly one Task})$$

$$x_{i,j} \in \{0, 1\}, \quad \forall 1 \leq i, j \leq N \quad (\text{All decision variables are binary})$$

Observation

A cost matrix

	Task 1	Task 2	Task 3
Agent 1	198	172	300
Agent 2	185	200	225
Agent 3	158	175	200

Question

Does uniformly adding a constant to a **row** or **column** of the **cost matrix** in a minimum-cost matching problem change the decision of optimal matching?

Observation

A cost matrix

	Task 1	Task 2	Task 3
Agent 1	198	172	300
Agent 2	185	200	225
Agent 3	158	175	200

Question

Does uniformly adding a constant to a **row** or **column** of the **cost matrix** in a minimum-cost matching problem change the decision of optimal matching?

No.

- Any perfect matching involves exactly one element from each row and each column.
- The cost of every perfect matching is shifted by the same constant.
- Therefore, the optimal matching remains unchanged.

The Hungarian algorithm

- The Hungarian algorithm efficiently solves the minimum-cost assignment problem optimally.
- It was developed by Harold Kuhn in 1955.
- The name of the algorithm reflects key ideas derived from the work of two Hungarian mathematicians:
 - ▶ Dénes König (1884–1944)
 - ▶ Jenő Egerváry (1891–1958)
- James Munkres showed in 1957 that the algorithm runs in polynomial time (originally $O(N^4)$).
- Since then, it has also been referred to as the Kuhn-Munkres algorithm (and sometimes the Munkres algorithm).
- Later refinements reduce the running time to $O(N^3)$.

Transforming the objective function to motivate the method/algorithm that follows

(Listen to the lecture to process the next few slides.)

$$z = \sum_{i=1}^N \sum_{j=1}^N c_{i,j} x_{i,j}$$

Introduce u_i and v_j as **dual variables** associated with each row and column, respectively.

$$= \sum_{i=1}^N \sum_{j=1}^N x_{i,j} \underbrace{(c_{i,j} - u_i - v_j + u_i + v_j)}_{=0}$$

$$= \sum_{i=1}^N \sum_{j=1}^N x_{i,j} (c_{i,j} - u_i - v_j) + \sum_{i=1}^N \sum_{j=1}^N x_{i,j} u_i + \sum_{i=1}^N \sum_{j=1}^N x_{i,j} v_j$$

$$= \sum_{i=1}^N \sum_{j=1}^N x_{i,j} (c_{i,j} - u_i - v_j) + \sum_{i=1}^N \left(u_i \sum_{j=1}^N x_{i,j} \right) + \sum_{j=1}^N \left(v_j \sum_{i=1}^N x_{i,j} \right)$$

but $\sum_{i=1}^N \sum_{j=1}^N x_{i,j} = 1$ and $\sum_{j=1}^N \sum_{i=1}^N x_{i,j} = 1$ (see slide #11)

$$\Rightarrow z = \sum_{i=1}^N \sum_{j=1}^N x_{i,j} \underbrace{(c_{i,j} - u_i - v_j)}_{=c'_{i,j}} + \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$$

... continued

Decomposition of the objective

$$z = \sum_{i=1}^N \sum_{j=1}^N x_{i,j} \underbrace{(c_{i,j} - u_i - v_j)}_{= c'_{i,j}} + \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$$

- The Hungarian (cost-matrix reduction) algorithm chooses u_i and v_j such that:
 - ▶ If $x_{i,j} = 1$ (Agent i assigned to Task j), then $c_{i,j} = u_i + v_j$ (i.e., $c'_{i,j} = 0$).
 - ▶ If $x_{i,j} = 0$, then $c_{i,j} \geq u_i + v_j$ (i.e., $c'_{i,j} \geq 0$).
- If such a choice is always made, clearly $\sum_{i=1}^N \sum_{j=1}^N x_{i,j} (c_{i,j} - u_i - v_j) = 0$.
- Therefore,

$$z = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j,$$

transforming the original (primal) minimization problem into a (dual) maximization problem involving $\{u_i\}$ and $\{v_j\}$. This is explained in the next slide.

Dual formulation of the min-cost assignment

Original (primal) formulation

Minimize

$$z = \sum_{i=1}^N \sum_{j=1}^N c_{i,j} x_{i,j}$$

subject to:

$$\sum_{i=1}^N x_{i,j} = 1 \quad \forall j$$

$$\sum_{j=1}^N x_{i,j} = 1 \quad \forall i$$

$$x_{i,j} \in \{0, 1\} \quad \forall i, j$$

Dual formulation

Maximize

$$z = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$$

subject to:

$$\underbrace{c_{i,j} - u_i - v_j}_{\text{(reduced cost } c'_{i,j})} \geq 0 \quad \forall i, j$$

which is same as saying, subject to:

$$u_i + v_j \leq c_{i,j} \quad \forall i, j$$

The equivalence is shown in the next slide.

Why is this dual formulation the same as the original?

Let M be **any feasible perfect matching** ($|M| = N$). The cost of M is:

$$\text{cost}(M) = \sum_{(i,j) \in M} c_{i,j}$$

Since $c_{i,j} \geq u_i + v_j$, we have:

$$\text{cost}(M) \geq \sum_{(i,j) \in M} (u_i + v_j)$$

Because M is a perfect matching, each row and each column appears exactly once, hence:

$$\text{cost}(M) \geq \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$$

Therefore, $\sum_{i=1}^N u_i + \sum_{j=1}^N v_j$ is a **lower bound** on the cost of any feasible perfect matching. If there exists a matching M^* such that equality holds, i.e.,

$$\text{cost}(M^*) = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j,$$

then M^* is an **optimal** perfect matching, since no feasible matching can have lower cost:

$$\text{cost}(M^*) \leq \text{cost}(M) \quad \forall M$$

In the dual formulation, since $u_i + v_j \leq c_{i,j}$, maximizing $\sum_{i=1}^N u_i + \sum_{j=1}^N v_j$ under these constraints gives equality as the tightest lower bound on the primal objective. This is analogous to the Max-Flow = Min-Cut theorem (FIT2004).

The Hungarian algorithm – Overview

- The Hungarian algorithm solves the dual formulation. (This is analogous to solving max-flow (dual) when the problem is min-cut (primal), from FIT2004.)
- Starting from the initial values $\{c'_{i,j}\} = \{c_{i,j}\}$ (i.e., $u_i = v_j = 0$), each **iteration** of the algorithm performs a progressive **reduction** of the cost matrix.
 - ▶ The reduction identifies admissible assignments (i, j) where $\underbrace{c_{i,j} - u_i - v_j}_{c'_{i,j}} = 0$.
- In each iteration, the algorithm **updates** u_i and v_j to **increase** the dual objective

$$z = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j,$$

while maintaining feasibility ($u_i + v_j \leq c_{i,j}$ for all i, j).

- The size of the current matching ($|M|$) is non-decreasing until it grows to become perfect ($|M| = N$).
- When a perfect matching is found, the algorithm **terminates correctly** with the minimum-cost assignment (by the result in Slide #17).
- The iterative steps of the Hungarian algorithm are described below with a running example.

Hungarian algorithm (cost-matrix reduction method)

Input: An $N \times N$ cost matrix defining the assignment problem (see example below).

Output: A perfect matching of size N with **minimum** total cost.

Example: Original cost matrix ($N = 5$)

	Task 1	Task 2	Task 3	Task 4	Task 5
Agent 1	11	7	10	17	10
Agent 2	13	21	7	11	13
Agent 3	13	13	15	13	14
Agent 4	18	10	13	16	14
Agent 5	12	8	16	19	10

Key steps of the Hungarian algorithm

- u_i values are initialized in Step 1; v_j values are initialized in Step 2.
- Steps 3-4 iteratively find augmenting assignments that increase the objective and update (u_i, v_j) when a bottleneck is encountered. These continue until a **perfect matching** is found (termination condition).

In later slides, “Agent” and “Task” labels in the matrix are ignored for conciseness.

Hungarian method – Step 1

(Assign a feasible set of u_i values...)

Step 1

In the cost matrix, for each row, find the row minimum and subtract it from every element of that row. This is same as initializing $u_i = \min(\text{row}_i)$.

Example; Step 1 (subtract by row minimum)

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	0
0	11	7	10	17	10
0	13	21	7	11	13
0	13	13	15	13	14
0	18	10	13	16	14
0	12	8	16	19	10



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	0
7	4	0	3	10	3
7	6	14	0	4	6
13	0	0	2	0	1
10	8	0	3	6	4
8	4	0	8	11	2

Hungarian method – Step 2

(Assign a feasible set of v_j values)

In the resultant matrix, for each column, find the column minimum and subtract it from every element of that column. Initialize $v_j = \min(\text{col}_j)$.

Example; Step 2 (subtract by column minimum)

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	0
7	4	0	3	10	3
7	6	14	0	4	6
13	0	0	2	0	1
10	8	0	3	6	4
8	4	0	8	11	2



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

After these assignments of u_i and v_j values, the algorithm iterates carrying out steps 3 and 4 (until termination).

Hungarian method – Step 3 (attempt assignments on $c'_{i,j} = 0$ cells)

Assign as many unambiguous rows and columns as feasible:

Step 3.1

- For each **row** i , if cell (i, j) is the **only** 0 valued cell available for assignment in that row, then assign $x_{i,j} = 1$ (i.e. Agent i is matched to Task j for this iteration), and cross out all other 0 cells in column j .
- If multiple 0s are available in any row, ignore that row.

Example; Step 3.1

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 0$$

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 2$$

Hungarian method – Step 3 continued (step 3.2)

Step 3.2 – Similar to Step 3.1, repeat for **columns** instead of rows:

- For each **column** j , if cell (i, j) is the **only** 0 valued cell available in that column, then assign $x_{i,j} = 1$ (i.e. Task j is matched to Agent i for this iteration), and cross out all other 0 cells in row i .
- If multiple 0s are available, ignore that column.

Example; Step 3.2

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 2$$

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

Notice all $c'_{i,j} = 0$ cells in the (right side) table are assigned or crossed out.

Caveat about Steps 3.1 and 3.2

- After completion of Step 3.2, it may happen that not all 0-valued cells in the matrix are crossed out (grey cells) or assigned (red cells).
- This indicates that there may be **multiple** possible optimal perfect matchings for the given problem.
- To resolve this, choose an **arbitrary assignment** among the uncrossed 0 cells, and then repeat Steps 3.1 and 3.2 until all 0 cells are either crossed out or assigned.

Note

This situation does not occur in this running example. It will encounter an example of this kind in your lab sheet.

Hungarian method – Step 3 continued (step 3.3)

Find **minimum vertex cover** over all **0** cells.

step 3.3 – procedure to find that minimum vertex cover

- 1 Mark all unassigned rows.

Example; Step 3.3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

→

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1	mark
7	4	0	3	10	2	
7	6	14	0	4	5	
13	0	0	2	0	0	
10	8	0	3	6	3	✓
8	4	0	8	11	1	✓

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

Hungarian method – Step 3 continued (step 3.3)

Find **minimum vertex cover** over all **0** cells.

step 3.3 – procedure to find that minimum vertex cover

- ➊ Mark all unassigned rows.
- ➋ If row is marked, for any 0 cell in that row, mark its corresponding column.

Example; Step 3.3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

→

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1	mark
7	4	0	3	10	2	
7	6	14	0	4	5	
13	0	0	2	0	0	
10	8	0	3	6	3	✓
8	4	0	8	11	1	✓

mark		✓				
------	--	---	--	--	--	--

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

Hungarian method – Step 3 continued (step 3.3)

Find **minimum vertex cover** over all **0** cells.

step 3.3 – procedure to find that minimum vertex cover

- ➊ Mark all unassigned rows.
- ➋ If row is marked, for any **0** cell in that row, mark its corresponding column.
 - ▶ Also, when a marked column has an assignment, then mark the corresponding row of that assignment.

Example; Step 3.3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

→

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1	mark
7	4	0	3	10	2	✓
7	6	14	0	4	5	
13	0	0	2	0	0	
10	8	0	3	6	3	✓
8	4	0	8	11	1	✓
mark		✓				

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

Hungarian method – Step 3 continued (step 3.3)

Find **minimum vertex cover** over all **0** cells.

step 3.3 – procedure to find that minimum vertex cover

- ➊ Mark all unassigned rows.
- ➋ If row is marked, for any 0 cell in that row, mark its corresponding column.
 - ▶ Also, when a marked column has an assignment, then mark the corresponding row of that assignment.
- ➌ Repeat 3.3.2 until no more rows/cols can be marked by the procedure.

Example; Step 3.3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

→

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1	mark
7	4	0	3	10	2	✓
7	6	14	0	4	5	
13	0	0	2	0	0	
10	8	0	3	6	3	✓
8	4	0	8	11	1	✓

mark		✓				
------	--	---	--	--	--	--

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

Hungarian method – Step 3 continued (step 3.3)

Find **minimum vertex cover** over all **0** cells.

step 3.3 – procedure to find that minimum vertex cover

- ➊ Mark all unassigned rows.
- ➋ If row is marked, for any 0 cell in that row, mark its corresponding column.
 - ▶ Also, when a marked column has an assignment, then mark the corresponding row of that assignment.
- ➌ Repeat 3.3.2 until no more rows/cols can be marked by the procedure.
- ➍ Cover all unmarked rows and marked columns. The number of covered row and columns represent the number of assignments in the reduced cost matrix.

Example; Step 3.3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

→

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1	mark
7	4	0	3	10	2	✓
7	6	14	0	4	5	
13	0	0	2	0	0	
10	8	0	3	6	3	✓
8	4	0	8	11	1	✓

mark		✓				
------	--	---	--	--	--	--

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

Hungarian method – Step 4 (Augment u_i and v_j values)

step 4

- If $|M| = N$, (i.e. perfect matching reached), **terminate** the algorithm. The current M will be the optimal assignment, and current $\sum_{i=1}^N u_i + \sum_{j=1}^N v_j$ value will be the min cost.
- else if $|M| < N$ (i.e. perfect matching **not** yet reached), then
 - ▶ Identify the minimum number θ among cells not covered.
 - ▶ For each row **not** covered, update $u_i = u_i + \theta$
 - ▶ For each col **covered**, update $v_j = v_j - \theta$
 - ★ The above two updates **implicitly** denotes the following adjustment to cells of the reduced cost-matrix:

$cost(i, j) = cost(i, j) - \theta$	if cell (i, j) covered by 0 lines.
$cost(i, j)$	same as before, if cell (i, j) is covered by 1 line.
$cost(i, j) = cost(i, j) + \theta$	if cell (i, j) is at the intersection of 2 lines.
 - ▶ Repeat from Step 3 until termination (slide #22).

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	0	0	0	1
7	4	0	3	10	2
7	6	14	0	4	5
13	0	0	2	0	0
10	8	0	3	6	3
8	4	0	8	11	1



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-1	0	0	1
8	3	0	2	9	1
7	6	15	0	4	5
13	0	1	2	0	0
11	7	0	2	5	2
9	3	0	7	10	0

$$\sum u_i + \sum v_j = 46, \text{ and } |M| = 3$$

$$\sum u_i + \sum v_j = 48, \text{ and } |M| = 3$$

Example – Second iteration, step 3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-1	0	0	1
8	3	0	2	9	1
7	6	15	0	4	5
13	0	1	2	0	0
11	7	0	2	5	2
9	3	0	7	10	0

$\sum u_i + \sum v_j = 48$, and $|M| = 3$

--steps
3.1, 3.2-->

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-1	0	0	1
8	3	0	2	9	1
7	6	15	0	4	5
13	0	1	2	0	0
11	7	0	2	5	2
9	3	0	7	10	0

$\sum u_i + \sum v_j = 48$, and $|M| = 4$

3	0	2	9	1
6	15	0	4	5
0	1	2	0	0
7	0	2	5	2
3	0	7	10	0

--steps 3.3
(1)-(3)-->

3	0	2	9	1	✓
6	15	0	4	5	
0	1	2	0	0	
7	0	2	5	2	✓
3	0	7	10	0	
	✓				

3	0	2	9	1	✓
6	15	0	4	5	
0	1	2	0	0	
7	0	2	5	2	✓
3	0	7	10	0	
	✓				

--steps 3.3 (4)-->

3	0	2	9	1	✓
6	15	0	4	5	
0	1	2	0	0	
7	0	2	5	2	✓
3	0	7	10	0	
	✓				

Example – Second iteration, step 4

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-1	0	0	1
8	3	0	2	9	1
7	6	15	0	4	5
13	0	1	2	0	0
11	7	0	2	5	2
9	3	0	7	10	0

$\sum u_i + \sum v_j = 48$, and $|M| = 4$



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-2	0	0	1
9	2	0	1	8	0
7	6	16	0	4	5
13	0	2	2	0	0
12	6	0	1	4	1
9	3	1	7	10	0

$\sum u_i + \sum v_j = 49$, and $|M| = 4$

Example – Third iteration step 3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-2	0	0	1
9	2	0	1	8	0
7	6	16	0	4	5
13	0	2	2	0	0
12	6	0	1	4	1
9	3	1	7	10	0

$$\sum u_i + \sum v_j = 49, \text{ and } |M| = 4$$

--steps
3.1, 3.2-->

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-2	0	0	1
9	2	0	1	8	0
7	6	16	0	4	5
13	0	2	2	0	0
12	6	0	1	4	1
9	3	1	7	10	0

$$\sum u_i + \sum v_j = 49, \text{ and } |M| = 4$$

2	0	1	8	0
6	16	0	4	5
0	2	2	0	0
6	0	1	4	1
3	1	7	10	0

--steps 3.3
(1)-(3)-->

2	0	1	8	0	✓
6	16	0	4	5	
0	2	2	0	0	
6	0	1	4	1	✓
3	1	7	10	0	✓
	✓			✓	

2	0	1	8	0	✓
6	16	0	4	5	
0	2	2	0	0	
6	0	1	4	1	✓
3	1	7	10	0	✓
	✓			✓	

--steps
3.3 (4)-->

2	0	1	8	0	✓
6	16	0	4	5	
0	2	2	0	0	
6	0	1	4	1	✓
3	1	7	10	0	✓
	✓			✓	

Example – Third iteration, step 4

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-2	0	0	1
9	2	0	1 θ	8	0
7	6	16	0	4	5
13	0	2	2	0	0
12	6	0	1	4	1
9	3	1	7	10	0

$$\sum u_i + \sum v_j = 49, \text{ and } |M| = 4$$



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-3	0	0	0
10	1	0	0	7	0
7	6	17	0	4	6
13	0	3	2	0	1
13	5	0	0	3	1
10	2	1	6	9	0

$$\sum u_i + \sum v_j = 50, \text{ and } |M| = 4$$

Example – Fourth iteration, step 3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-3	0	0	0
--	---	----	---	---	---

10	1	0	0	7	0
7	6	17	0	4	6
13	0	3	2	0	1
13	5	0	0	3	1
10	2	1	6	9	0

$$\sum u_i + \sum v_j = 50, \text{ and } |M| = 4$$

--steps

3.1,

3.2-->

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-3	0	0	0
--	---	----	---	---	---

10	1	0	0	7	0
7	6	17	0	4	6
13	0	3	2	0	1
13	5	0	0	3	1
10	2	1	6	9	0

$$\sum u_i + \sum v_j = 50, \text{ and } |M| = 4$$

1	0	0	7	0
6	17	0	4	6
0	3	2	0	1
5	0	0	3	1
2	1	6	9	0

--steps 3.3

(1)-(3)--> →

1	0	0	7	0	✓
6	17	0	4	6	✓
0	3	2	0	1	
5	0	0	3	1	✓
2	1	6	9	0	✓
	✓	✓		✓	

1	0	0	7	0	✓
6	17	0	4	6	✓
0	3	2	0	1	
5	0	0	3	1	✓
2	1	6	9	0	✓
	✓	✓		✓	

--step 3.3 (4)-->

1	0	0	7	0	✓
6	17	0	4	6	✓
0	3	2	0	1	
5	0	0	3	1	✓
2	1	6	9	0	✓
	✓	✓		✓	

Example – Fourth iteration, step 4

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-3	0	0	0
10	1 θ	0	0	7	0
7	6	17	0	4	6
13	0	3	2	0	1
13	5	0	0	3	1
10	2	1	6	9	0

$\sum u_i + \sum v_j = 50$, and $|M| = 4$



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-4	-1	0	-1
11	0	0	0	6	0
8	5	17	0	3	6
13	0	4	3	0	2
14	4	0	0	2	1
11	1	1	6	8	0

$\sum u_i + \sum v_j = 51$, and $|M| = 4$

Example – Fifth iteration, step 3

$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-4	-1	0	-1
11	0	0	0	6	0
8	5	17	0	3	6
13	0	4	3	0	2
14	4	0	0	2	1
11	1	1	6	8	0

$$\sum u_i + \sum v_j = 51, \text{ and } |M| = 4$$



$\frac{v_j \rightarrow}{u_i \downarrow}$	0	-4	-1	0	-1
11	0	0	0	6	0
8	5	17	0	3	6
13	0	4	3	0	2
14	4	0	0	2	1
11	1	1	6	8	0

$$\sum u_i + \sum v_j = 51, \text{ and } |M| = 5$$

Now $|M| = 5$ matches/assignments have been made, implies perfect matching has been achieved. Algorithm **TERMINATES**.

The assignments (the cells highlighted in red) correspond to (Agent, Task) cells:
 (1, 1), (2, 3), (3, 4), (4, 2), (5, 5).

The min cost assignment is 51.

Example – Interpreting the result

Final reduced cost matrix ($c'_{i,j}$)(upon termination)

$\frac{v_j \rightarrow}{u_i \downarrow}$		0	-4	-1	0	-1
11		0	0	0	6	0
8		5	17	0	3	6
13		0	4	3	0	2
14		4	0	0	2	1
11		1	1	6	8	0

Original Cost matrix ($c_{i,j}$) (with assignments overlaid):

11	7	10	17	10
13	21	7	11	13
13	13	15	13	14
18	10	13	16	14
12	8	16	19	10

- $\sum_{i=1}^N u_i + \sum_{j=1}^N v_j = 11 + 8 + 13 + 14 + 11 + 0 - 4 - 1 + 0 - 1 = 51$.
- The optimal matching $M = \{(1, 1), (2, 3), (3, 4), (4, 2), (5, 5)\}$.
- Individual assignment cost:

$$c_{1,1} = 11 \quad \equiv \quad c'_{1,1} + u_1 + v_1 = 0 + 11 + 0 = 11$$

$$c_{2,3} = 7 \quad \equiv \quad c'_{2,3} + u_2 + v_3 = 0 + 8 - 1 = 7$$

$$c_{3,4} = 13 \quad \equiv \quad c'_{3,4} + u_3 + v_4 = 0 + 13 + 0 = 13$$

$$c_{4,2} = 10 \quad \equiv \quad c'_{4,2} + u_4 + v_2 = 0 + 14 - 4 = 10$$

$$c_{5,5} = 10 \quad \equiv \quad c'_{5,5} + u_5 + v_5 = 0 + 11 - 1 = 10$$

- $\sum_{(i,j) \in M} C_{i,j} = 11 + 7 + 13 + 10 + 10 = 51$
- Notice $\sum_{i=1}^N u_i + \sum_{j=1}^N v_j = \sum_{(i,j) \in M} C_{i,j}$ (Refer slide 17)

Handling problem instances in non-standard forms

- We studied the Hungarian algorithm that starts with a **cost matrix** and minimizes the total cost of a perfect matching.
- Sometimes, problem instances come defined over a **weight** matrix, where the goal is to **maximize** total weight. This can be converted to a **cost minimization** problem by replacing each weight $w_{i,j}$ with $c_{i,j} = -w_{i,j}$.
- Sometimes, the cost/weight matrix is rectangular, $M \times N$. To apply the algorithm, we square up the matrix by adding dummy rows or columns. The dummy entries should be chosen so they do not affect the optimal assignment (e.g., zero for **cost minimization**).

Time complexity **not-examinable**

For an $N \times N$ matrix of costs:

- Steps 1 and 2 requires worst case $O(N^2)$ effort.
- In each iteration, at least one previously non-0 cell reduces to 0. Hence, the total number of iterations is worst case $O(N^2)$ – in practice this is $O(N)$.
- In each iteration:
 - ▶ Steps 3 requires $O(N^2)$ (with supporting data structures):
 - ★ steps 3.3.2-3.3.3 requires $O(N^2)$ effort, as there are at most $O(N)$ unassigned rows to mark, and in each marked row at most $O(N)$ cols with 0s to mark, and each col can only have one assignment whose row to mark.
 - ▶ Step 4 takes $O(N^2)$ effort.
- Total worst-case complexity is $O(N^4)$.

Summary

- The Hungarian Algorithm solves the bipartite matching problem under a cost/weight function.
- Note: A closely-related algorithm graph-based notions of shortest alternating path to iteratively augment the matching (over 'equality' graphs). With some optimizations, this algorithm is guaranteed to run in $O(N^3)$ worst-case. Many steps of that algorithm mirror those of the Hungarian (except the augmentation step).
- However, in practice (on average), the performance of the Hungarian (cost matrix reduction) algorithm and shortest augmenting path method are nearly equivalent.

--o0o--

THE END

--o0o--